Embedded Computing System Coursework Report

Group 35

Art Younis Hubsey Damian

I. TASK 6

A. Question 1

Role: the FPU enables support for IEEE-754 single-precision floating point operations. The FPU adds an extended set of floating point registers and instructions to the standard architecture and provides integration of floating point hardware into the CPU. Since the FPU instructions are extensions of the standard C28x instruction set, most instructions operate in one or two pipeline cycles and some can be done in parallel this shows to be useful and essential in real time control applications where fast timing is important.

B. Question 2

To design this dual-core motor control system, we can assign one CPU core to manage real-time motro control (PWM generation, current sensing) and the other CPU for high-lvel tasks like trajectory planning. Then we can use each core's CLA for computationally intensive tasks like current loops or transformations (Clarke/Park). This parallelism could reduces latency, maximizes efficiency, and improves performance.

C. Question 3

Automating build, test, and deployment for the TMS320F28379D microcontroller using CI tools enables a more reliable and efficient workflow. By using a CI/CD pipeline, developers can ensure rapid feedback, thorough testing, and deployment of verified code to the microcontroller.

Code Composer Studio includes **ccstudio** and **gmake** command line tools for building projects. Additionally, by adding CCS to your system's path, CCS CLI commands can be easily accessed from any location in the CI/CD pipeline.

It is possible to integrate with a CI server (e.g., Jenkins, GitHub Actions, or GitLab CI) to automate the build process in response to code changes.

Code Composer Studio enables testing embedded code for both software and hardware. Interactions with physical components can be tested. Unit tests can be implemented with a unit testing framework – as CCS allows for the CI pipeline to be configured, doing unit tests separately, or tests without requiring hardware.

For testing, the CCS Scripting Console can be used to automate the firmware flashing process. The CI server can be configured to run deployment script only if all previous stages are successful.

D. Question 4

1. Initialize with Device_init() and Device_initGPIO() to set up the clock and GPIO peripherals
2. Configure pin type using GPIO_setPadConfig(pin, type) (e.g., GPIO_PIN_TYPE_STD for output, GPIO_PIN_TYPE_PULLUP for input).
3. Set direction using GPIO_setDirectionMode(pin, mode) (GPIO_DIR_MODE_OUT for output, GPIO_DIR_MODE_IN for input).
4. Optionally, set multiplexing or filtering as needed

E. Question 5

Submit the project file (Group43_Task6Q5) together with other codes.

The submitted code successfully implements an interrupt-based GPIO system on TMS320F28379D where some action is performed in response to the GPIO input (Push Button). Button debounce is used to ensure a correct response to a button push.

The system begins by setting up the device with "Device_init()", "Interrupt_initModule()", and "Interrupt_initVectorTable()". Next, GPIO is configured, this includes setting up an interrupt for the desired GPIO pin. Lastly, the interrupt is registered to the interrupt handler and is enabled.

The main loop decrements the debounce counter if it has been set, otherwise it infinitely idles. The interrupt handler will perform the action and reset the debounce counter, only when the counter is at 0 (completed the wait).

Due to mechanical bounces caused by the physical press, fluctuations may occur causing the interrupt to falsely fire. To rectify this problem, a software-based solution for a debounce has been implemented. In the code, a global variable is used to track a forced wait period (debounceCounter) - of around 10ms. Whenever an interrupt is called legitimately, the action is performed, and the counter is reset to its waiting period. If the interrupt is called again during the wait period, the action will not perform, and the counter will not reset.

F. Question 6

Submit the project file (Group43_Task6Q6) together with other codes.

G. Question 7

Submit the project file (Group43_Task6Q7) together with other codes.

H. Question 8

In the TMS320F28379D microcontroller, GPxMUX1/2 are multiplexing control registers, used to control pin functions. GPxGMUX serves as general mux control, it also controls pin functions using GPxMUX1/2. They are both required for mapping specific GPIO pins. GPxMUXn takes a bit-pair that corresponds to a single GPIO pin. The GPxGMUXn register functions as a multiplexer, also taking a bit-pair, and when used in conjunction with GPxMUXn, pin selection can be refined.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

The TMS320F28379D microcontroller has 97 GPIO pins, which are accessed through six GPIO ports: GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, and GPIOF. Therefore, when using GPxMUXn and GPxGMUXn, a port number must be given and another number (1 or 2) to select GPIOn for a specific peripheral.

An additional number is required because with two bit-pairs in a multiplexer, there are in total 16 combinations for pin selection. With ports containing 32 pins, the first 16 and the second 16 pins must be selected. For each GPIO pin, in GPAMUX1 two bits are used for selecting a particular option, mainly for assigning an alternate peripheral function.

I. Question 9

Peripheral interrupts in the TMS320F28379D are managed using the PIE (Peripheral Interrupt Expansion) module, which organizes up to 96 interrupts into 12 groups, each with 8 interrupts. The PIE enables efficient interrupt handling by vectorizing interrupts, allowing each to have its own ISR (Interrupt Service Routine), a special function executed in response to an interrupt.

Interrupts are prioritized within groups, with lower-numbered interrupts having higher priority. Groups with higher priority interrupt the lower ones, ensuring critical tasks are handled first. The PIE sends interrupts to the CPU and manages their acknowledgment, making it useful for real-time tasks like motor control.

J. Question 10

An NMI is a non-maskable interrupt which means the processor cannot ignore it unlike a maskable interrupt which can be disabled/ignored by the processor.

Each CPU has its own NMI module, an incoming NMI sets a status bit in the NMIFLG register and then starts the NMI watchdog counter. The SYSCLK clocks the counter and if it has the same vale as the NMIWDPRD register then it triggers an NMI watchdog rest. To prevent this the NMI handler must clear the flag but using the NMIFLGCLR register.

Handling: once all flag bits are clear the NMIINT bit in the NMIFLG register may also be cleared to allow future NMIs to be taken.

How the NMI handler is configured:

THE NMI module is enabled by the boot ROM during the start-up process. To respond to NMIs an NMI handler vector must be written to the PIE vector table.

Challenges with NMI:

Unlike a normal interrupt an NMI is non-maskable this means it cannot be ignored and the processor must respond as soon as possible to the interrupt request. This means at times other critical tasks would be disrupted so the processor can handle the NMI.

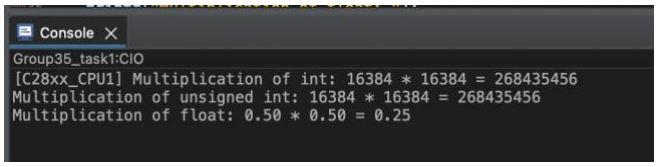
Another challenge is that NMI cannot be disabled this means that there is nothing that stops the NMI unlike maskable interrupts which can be temporarily disabled, this means the systems doesn't have the ability to control when the interrupts happen so they can happen at times when the CPU is carrying out critical operations and will cause the CPU to forcefully move to handling the NMI and pausing the other critical task it was doing before.

How the systems ensures NMI sources are managed correctly:

In order to manage the NMI sources the system uses NMIFLG register that record the status of an incoming NMI, it uses an NMI watchdog to have the NMI solved in a certain time period and NMIFLGCLR clears the flag in the NMIFLG once the interrupt has been handled signalling that the NMI has been resolved. This makes sure that the NMI sources are managed correctly.

II. TASK 1

Submitted project file 'Group35_Task1'.



III. TASK 2

A. Deliverable 1

Submitted project file 'Group35_Task2'.

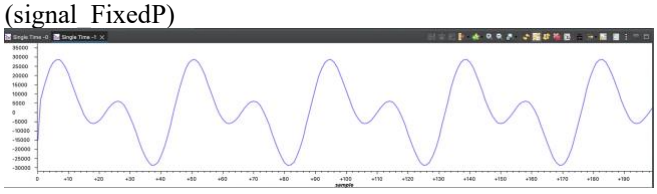
B. Deliverable 2

The FIR filter removes unwanted parts of a signal by calculating a weighted sum of the current and past inputs (convolution). The weights, called coefficients (firCoeffs[]), control which parts of the signal are kept or reduced.

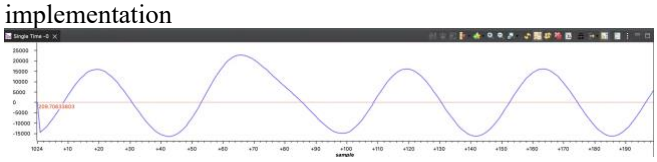
The filtering is done in the filter1() function. Here, each new input sample is added to a buffer (filtbuff1[]), shifting older samples down. The function then multiplies each sample in the buffer by its weight and sums them up. This sum is the filtered output. Organizing the filtering logic in filter1() ensures modularity and focuses the convolution process in one function, making it easier to debug, maintain, and test. It simplifies signal processing and enables clear separation of tasks within the code

C. Deliverable 3

The following is the original signal represetantion (signal_FixedP)



The following shows the fixed-point representation of the filtered signal after applying the FIR filter (Buffer1Fixed). The attenuation of high-frequency components is visible, confirming the filter's low-pass behavior and its proper implementation



IV. TASK 3

A. Deliverable 1

Submitted project file 'Group35_Task3'.

B. Deliverable 2

The Goertzel algorithm is a computationally efficient way of detecting specific frequencies. It is a technique in digital signal processing for evaluating individual terms of the discrete Fourier transform. It is ideal for dual-tone multi-frequency detection where a small number of fixed frequencies are used.

As seen in the function **Generate_tones**, a switch-case block selects two frequencies based on the input character (0-9, A-D, *, #). Additionally, in **main**, a set of the standard DTMF frequencies are given, allowing for the Goertzel algorithm to work ideally.

The Goertzel algorithm works by first calculating the omega, cosine, and sine values for each target frequency. Next, values for the filter values q0, q1, and q2 are recursively calculated. These values are then used to calculate the magnitude for the current frequency. Finally, the global array magnitudes can be used for printing or further processing.

For our implementation to work on the C28379D processor, it was imperative for us to ensure that memory is validly allocated. A command file (.cmd) can map sections such as '.text,' '.const,' '.data,' '.bss,' and '.stack' to memory sections such as RAM and Flash.

Our solution was to change the mapping for the .text section. The code and variables used in the Goertzel algorithm resulted in the .text section requiring too much memory for the previous mapping. We mapped it to RAMGS1, RAMGS2, RAMGS3, and RAMGS4 from Page 1.

V. TASK 4

A. Deliverable 1

Submitted project file 'Group35_Task4'.

B. Deliverable 2

Using the project files given from the lab, modifications had to be made for both CPUs so that the contents of 'shared_array_C1' could be sent from CPU 1 to CPU 2.

The implementation requires the synchronization of both CPUs - so that CPU 1 can send data when CPU 2 is ready. The IPC17 flag is used to indicate when CPU 2 is ready for the next value. CPU 1 waits for IPC17 using a while loop in-between sending data using the IPCSENDDATA register.

Values from 'shared_array_C1' are sent by using a while loop where the integer variable 'i' tracks the current index to use. Once data is sent using IPCSENDDATA, the flag IPC1 is set, notifying CPU 2 that new data is available. An interrupt is sent for CPU 2, where it can process the data. CPU 2 sets the IPC17 flag to signal that it is currently processing the data, it is cleared after acknowledgment. CPU 1 waits for IPC 17 to be cleared, then it can move on to the next value of 'shared_array_C1' and repeat the process.

CPU 2 acts as the receiver, IPC1 is used for interrupts, otherwise CPU 2 waits in an endless loop in main (after initialization). After processing data from the interrupt, IPC1 is used to acknowledge CPU 1 – and IPC17 signals readiness to CPU 1. Additionally, PIE group 1 is acknowledged to enable further interrupts from CPU 1.

To summarize, both CPUs begin by initializing their corresponding IPC registers and interrupt vectors. CPU 2 then sets IPC17 to signal CPU 1 that it is ready. CPU 1 can then send a value using IPCSENDDATA and signal CPU 2 using IPC1. As CPU 2's ISR is triggered, the IPCRECVDATA register is used to receive the data – which is then validated and stored in 'shared_array_C2'. Afterwards, CPU 2 acknowledges the reception by clearing IPC1 and setting IPC17. As CPU 1 has been waiting for IPC17, it can then clear it and proceed to the next value. This is repeated for all 20 values in 'shared_array_C1'.

VI. TASK 5

A. Deliverable 1

Submitted project file 'Group35_Task5'.

B. Deliverable 1

Introduction:

The direct memory access(DMA) provides a method of transferring data between peripherals and memory without intervention from the CPU, thereby freeing up bandwidth for other system functions. Additionally the DMA has the capability to "ping-Pong" data between buffers. these features are useful for structuring data into blocks for CPU processing. This project demonstrates this by using the DMA(Direct Memory Access) engine on our C28379D microcontroller in order to transmit data from array_1 to array_2.

Our goal for this project was to be able to transmit data from array_1 to array_2 without CPU involvement and only using the DMA engine.

We used the C28379D microcontroller for hardware and for our IDE we used code composer studio.

Code explanation:

```
//
#include "driverlib.h"
#include "device.h"

//
// DMA data sections
//
#pragma DATA_SECTION(array_1, "ramgs0");  // map the TX data to memory
#pragma DATA_SECTION(array_2, "ramgs1");  // map the RX data to memory

//
// Defines
//
#define BURST       8       // write 8 to the register for a burst size of 8
#define TRANSFER    16      // [(MEM_BUFFER_SIZE/(BURST)]

//
// Globals
//
uint16_t array_1[100] = {
 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
 21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
 51, 52, 53, 54, 55, 56, 57, 58, 59, 60,
 61, 62, 63, 64, 65, 66, 67, 68, 69, 70,
 71, 72, 73, 74, 75, 76, 77, 78, 79, 80,
 81, 82, 83, 84, 85, 86, 87, 88, 89, 90,
 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
};;   // Send data buffer
uint16_t array_2[100];   // Receive data buffer
volatile uint16_t done;

```

Array_1 and array_2 are mapped to ramgs0 and ramgs1 memory sections by using Pragma directive, we also define our BURST and TRANSFER size which is needed when doing the dma transfer. We then initialised our array_1 and array_2 giving array_1 100 elements of which we want to transfer all to array_2 which we have also said that it must have space to take 100 elements.

```c
void initDMA()
{
    //
    // Refer to dma.c for the descriptions of the following functions.
    //

    //
    //Initialize DMA
    //
    DMA_initController();

    const void *destAddr;
    const void *srcAddr;
    srcAddr = (const void *)array_1;
    destAddr = (const void *)array_2;

    //
    // configure DMA CH6
    //
    DMA_configAddresses(DMA_CH6_BASE, destAddr, srcAddr);
    DMA_configBurst(DMA_CH6_BASE,8,1,1);
    DMA_configTransfer(DMA_CH6_BASE,100,1,1);
    DMA_configMode(DMA_CH6_BASE,DMA_TRIGGER_SOFTWARE, DMA_CFG_ONESHOT_DISABLE);
    DMA_setInterruptMode(DMA_CH6_BASE,DMA_INT_AT_END);
    DMA_enableTrigger(DMA_CH6_BASE);
    DMA_enableInterrupt(DMA_CH6_BASE);
}
```

We then create a initDMA function which in this function we initialise the DMA using DMA_initcontroller() and we map the source address and destination address to array_1 and array_2. We then configured the DMA channel, the DMA channels Burst settings ,transfer settings and its trigger mode, the DMA config burst has a burst size of 8 meaning there is 8 elements per burst and the DMA config transfer size function sets the transfersize which for us is 100 elements. The transfer is triggered by the DMAconfig trigger function.

```c
//
__interrupt void dmaCh6ISR(void)
{
    uint16_t i;

    DMA_stopChannel(DMA_CH6_BASE);
    // ACK to receive more interrupts from this PIE group
    EALLOW;
    Interrupt_clearACKGroup(INTERRUPT_ACK_GROUP7);
    EDIS;

    for( i = 0; i < 100; i++ )
    {
        //
        // check for data integrity
        //
        if (array_2[i] != i)
        {
            error();
        }
    }

    done = 1; // Test done.
    return;
}

//
// End of file
```

We are using the dma channel 6 and it is started by "DMA_startChannel(DMA_CH6_BASE)" which starts our dma channel.we used "done" to see if the ISR was triggered and executed correctly.if done = 1 then it was complete and done = 0 if not.

We also use the DMACH6ISR which is an ISR(interrupt service routine) this veries the data integrity and signals when the transfer has completed.

Testing and results:

To verify that our code works and the DMA transfer works we compiled the code and then tested it using the debug function, in our expressions page:

Expression	Type	Value	Address
array_1	unsigned int[100]	[1,2,3,4,5...]	0x0000C000@Data
array_2	unsigned int[100]	[0,0,0,0,0...]	0x0000D000@Data
done	unsigned int	0	0x0000AAA1@Data

We add array_1 and array_2 to our expression to monitor the transfer of data and also we had the variable "done" which would indicate that the code works without any errors and the transfer is complete by returning 1 in the expressions page.

Variables | Expressions | Registers

Expression	Type	Value	Address
array_1	unsigned int[100]	[0,1,2,3,4...]	0x0000C000@Data
array_2	unsigned int[100]	[0,1,2,3,4...]	0x0000D000@Data
done	unsigned int	1	0x0000AAA1@Data

Once we ran the code I saw that array_1 and array_2 now had the same contents inside of them, the 100 elements in array_1 and was also now in array_2 and also that done = 1, this let us know that it has worked and the DMA transfer was complete.

REFERENCES

[1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955. *(references)*

[2] J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

[3] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.